

# GPU 101

How a machine for pictures learned to compute  $A + B$

Minjae Gwon  
POSTECH  
September 8, 2026

오늘은 그림을 그리던 기계가 어떻게  $A$  더하기  $B$ 를 계산하게 됐는지 이야기해보겠습니다. GPU의 사양표에 있는 숫자를 외우기보다, 왜 이런 구조가 필요했는지부터 따라가보려고 합니다.

출발점은 게임 화면입니다. 수많은 위치에 같은 계산법을 적용하던 구조에서 그림이라는 의미를 벗겨내면, 수많은 데이터에 같은 함수를 적용하는 계산기가 보입니다. 먼저 이 변화가 어떻게 일어났는지 보고, warp와 SM이라는 실행 구조를 이해한 다음, 마지막에는 PyTorch의 한 줄이 실제 하드웨어에 도착하는 과정을 연결하겠습니다.

# 1. Motivation

- Why GPUs matter
- Parallel work in pictures

# 2. From pictures to programs

- Rendering and shaders
- GPGPU and CUDA

# 3. The execution engine

- Warps and SIMT
- SMs and GPU organization

# 4. Following A + B

- From tensors to threads
- Scheduling and memory
- Utilization and completion

## Contents

발표는 네 부분입니다. 먼저 GPU를 이해하면 무엇이 보이는지 동기를 짚고, 그래픽 장치가 범용 계산 플랫폼으로 이어진 과정을 보겠습니다. 그다음 warp와 SM을 중심으로 실행 엔진을 이해합니다. 마지막으로 PyTorch의 덧셈 한 줄에서 출발해 데이터 이동, 작업 배치, 실제 명령 실행과 완료 시점까지 따라가겠습니다.

---

## 1. Motivation

- Why GPUs matter
- Parallel work in pictures

---

## 2. From pictures to programs

---

## 3. The execution engine

---

## 4. Following A + B

---

## Motivation

먼저 GPU를 왜 이해하면 좋은지부터 짚겠습니다. 프레임워크와 모니터링 도구에서 보이는 현상을 설명할 기본 그림을 만드는 것이 목표입니다. 이어서 그림을 병렬로 만드는 시연으로 직관을 잡겠습니다.

## What should make more sense after this talk?

- **The GPU you observe**

What are VRAM and all those execution resources?

- **The frameworks you use**

What sits below a PyTorch or Hugging Face operation?

- **The platform NVIDIA built**

How did graphics hardware become a general compute platform?

Pictures → Execution engines →  $C = A + B$

## What should make more sense after this talk?

GPU를 쓰다 보면 nvidia-smi를 켜서 메모리 사용량을 보거나, 모델을 올렸는데 왜 기다리는지 궁금해지는 순간이 있습니다. PyTorch나 Hugging Face에서는 몇 줄로 끝나는 작업이 아래에서는 어떻게 실행되는지도 잘 보이지 않습니다.

오늘 가져갈 것은 그 아래를 설명할 수 있는 기본 그림입니다. VRAM에 있는 데이터와 GPU 안의 실행 자원은 어떤 관계인지, 프로그램의 thread와 물리적인 연산기는 왜 다른지 알아보겠습니다. NVIDIA가 그래픽 장치를 계산 플랫폼으로 확장한 과정도 같은 흐름에서 이해할 수 있습니다. 구체적인 성능 튜닝이나 매출 분석에 들어가기 전에 필요한 개념을 함께 만드는 시간이라고 생각하면 됩니다.

## One image. Very different ways to paint it.



Parallel work becomes easy to see when the output is a picture.

### CPU vs. GPU

One paintball at a time.

Many paintballs together.

► [Watch the demo](#)

## One image. Very different ways to paint it.

먼저 MythBusters가 NVIDIA 행사에서 보여준 시연을 보겠습니다. 사진이나 오른쪽 링크를 누르면 영상이 열립니다. 한쪽은 페인트볼을 하나씩 쏘고, 다른 쪽은 많은 페인트볼을 함께 쏘서 그림을 만듭니다. 발표 시간이 허용되면 여기서 영상을 재생하고 결과를 함께 보겠습니다.

이 시연에서 가져올 것은 여러 위치의 작업을 동시에 진행할 수 있다는 직관입니다. 실제 CPU가 언제나 하나씩만 계산한다거나, GPU가 어떤 작업이든 무조건 빠르다는 뜻은 아닙니다. 이제부터는 왜 그림을 그리는 작업에 이런 병렬성이 많았는지를 보겠습니다. 참고로 링크의 현재 업로더는 HOSTKEY이고, 시연 자체는 2008년 NVIDIA NVISION 행사에서 나온 것입니다.

NVIDIA NVISION 08 / MythBusters demo · video re-upload: HOSTKEY

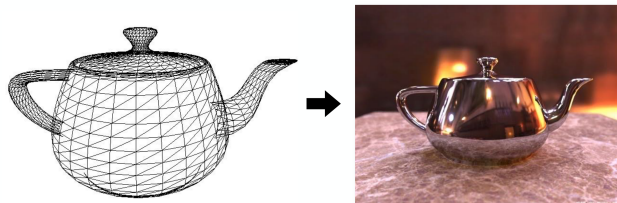
- Rendering and shaders
- GPGPU and CUDA

## From pictures to programs

이제 그래픽에서 범용 계산으로 이어지는 과정을 보겠습니다. 많은 픽셀에 같은 셰이더를 적용하던 구조에서 출발해, 숫자가 꼭 색일 필요는 없다는 발견과 CUDA가 바꾼 프로그래밍 방식을 연결하겠습니다.

## The original deadline: draw the next frame

### 3D rendering



**Input: description of a scene:**

3D surface geometry (e.g., triangle mesh)  
surface materials, lights, camera, etc.

**Output: image of the scene**

Image credit: Henrik Wann Jensen

**Geometry + materials + lights + camera → an image**

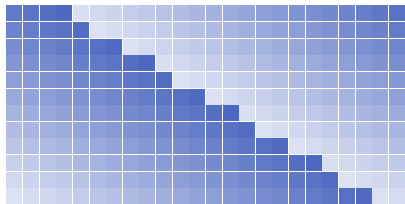
## The original deadline: draw the next frame

게임 속에서 햇빛이 비치는 호숫가를 걷는다고 생각해봅시다. 카메라가 움직이면 물결의 모양, 나무에 가려지는 부분, 수면에 비치는 빛이 모두 달라집니다. 컴퓨터가 처음부터 완성된 그림을 갖고 있는 것은 아닙니다. 물체의 삼각형들, 재질, 조명, 카메라 위치를 가지고 다음 화면을 계산해야 합니다.

이 작업이 렌더링입니다. 왼쪽에는 표면의 모양을 나타내는 기하 정보가 있고, 오른쪽에는 빛과 재질을 반영한 이미지가 있습니다. GPU는 원래 이런 3차원 그래픽을 빠르게 처리하기 위해 발전했습니다. 그런데 게임에서는 한 장만 잘 그리면 되는 것이 아니라, 다음 장도 정해진 시간 안에 계속 만들어야 합니다.

Stanford CS149, Fall 2025 · slide 5 · image: Henrik Wann Jensen

## Millions of positions. The same kind of calculation.



Different positions, different inputs

**1920 × 1080**

output pixels per frame

× **60 frames / second**

≈ **124 million**

output pixels per second

**The opportunity is repeated work across independent data.**

## Millions of positions. The same kind of calculation.

1920 곱하기 1080 화면에는 약 200만 개의 픽셀이 있습니다. 이것을 초당 60장 만들면 매초 약 1억 2천4백만 개의 출력 픽셀을 만들어야 합니다. 이것은 연산 횟수가 아니라 출력 픽셀 수입니다. 하나의 픽셀을 정하기 위해서는 훨씬 많은 계산이 필요할 수 있습니다.

여기서 중요한 것은 단순히 숫자가 크다는 사실만이 아닙니다. 수면의 이 지점에서도 빛의 반사를 계산하고, 바로 옆 지점에서도 비슷한 계산을 합니다. 위치와 방향 같은 입력은 다르지만 적용할 계산법은 같을 수 있습니다. 서로 다른 위치에 대해 비슷한 계산을 아주 많이 한다는 성질이 GPU가 활용할 수 있는 병렬성의 출발점입니다.

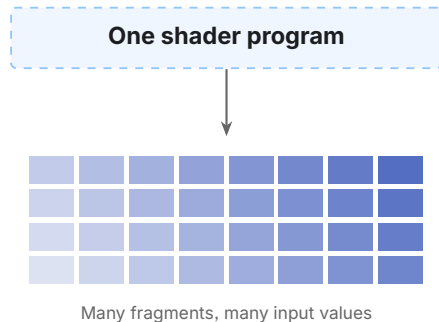
## A shader is a program for a graphics stage

### ONE FRAGMENT'S WORK

```
base    = read_surface_color(uv)
light   = max(dot(normal, light_dir), 0)
output  = base * light
```

- Programmable behavior replaces a fixed formula.
- A fragment is a candidate contribution to a pixel.

**Write how one surface point is shaded; apply it across the surface.**



Concept: Stanford CS149, Fall 2025 · slides 11–14 · simplified pseudocode

## A shader is a program for a graphics stage

물과 벽돌을 언제나 같은 공식으로 그릴 수는 없겠죠. 초기 그래픽 하드웨어는 정해진 기능을 빠르게 처리하는 성격이 강했지만, 이후에는 파이프라인의 일부를 프로그래머가 작성한 코드로 바꿀 수 있게 됐습니다. 이것이 프로그래머블 셰이더입니다. 셰이더는 물리적인 연산기 이름이 아니라 그래픽 처리의 특정 단계에서 실행되는 프로그램입니다.

정점 셰이더는 꼭짓점의 위치를 계산하고, 여기서 보는 프래그먼트 셰이더는 표면 조각의 색을 계산합니다. 표면색을 읽고, 빛을 얼마나 정면으로 받는지 구한 다음, 둘을 곱하는 예시입니다. 같은 프로그램을 여러 위치에 적용하되 입력이 달라서 결과색도 달라집니다. 프래그먼트는 최종 픽셀 그 자체는 아니며, 가려짐 검사 등의 과정을 거쳐 화면에 기여할 수 있는 조각이라고 생각하면 됩니다.

## Do those numbers have to be colors?

### THE GRAPHICS INTERPRETATION



### THE COMPUTE INTERPRETATION



### GPGPU: General-Purpose Computing on GPUs

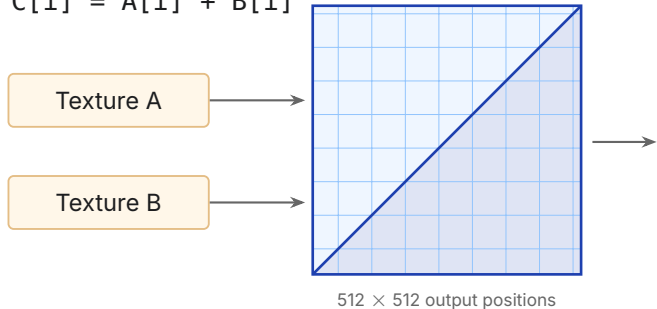
## Do those numbers have to be colors?

이 구조를 보던 연구자가 이런 생각을 했다고 해봅시다. 저 숫자가 꼭 색깔이어야 할까? GPU는 입력값을 사람이 물의 색이라고 부르는지, 온도라고 부르는지 신경 쓰지 않습니다. 숫자를 읽고 계산한 다음 결과를 저장할 뿐입니다.

위쪽은 그래픽의 언어입니다. 각 위치의 입력을 셰이더에 넣어 출력색을 만듭니다. 아래쪽에서는 이름만 바꿨습니다. 각 데이터 포인트의 입력을 함수에 넣어 결과값을 만듭니다. 기존 기계를 새로 발명한 것이 아니라, 원래 하던 일에서 그림이라는 의미를 벗겨낸 것입니다. 이런 식으로 많은 데이터에 같은 계산을 병렬로 적용할 수 있다면 그래픽이 아니어도 GPU를 활용할 수 있습니다. 이것이 초기 GPGPU의 핵심 관점입니다.

## To add two arrays, first draw two triangles

$$C[i] = A[i] + B[i]$$



- 1 Set the output image size.
- 2 Cover it with two triangles.
- 3 Add values in a fragment shader.

Read the image back as an array.

**A draw call to the GPU; an array operation to the researcher.**

## To add two arrays, first draw two triangles

발견과 실제 사용 사이에는 재미있는 장애물이 있었습니다. 우리가 하고 싶은 것은 A와 B의 같은 원소를 더해서 C에 쓰는 것뿐인데, GPU에 일을 시키는 통로가 그래픽 API였습니다. 그래서 계산을 그림 그리기로 번역해야 했습니다.

512 곱하기 512개의 원소를 처리하려면 먼저 출력 이미지 크기를 그렇게 설정합니다. 입력 배열은 원래 표면의 무늬를 저장하던 텍스처에 넣습니다. 그다음 출력 영역을 정확히 덮는 삼각형 두 개를 그립니다. 그러면 각 위치에 프래그먼트 셰이더가 실행되고, 셰이더는 색을 만드는 대신 A와 B의 값을 읽어 더합니다. 출력된 이미지를 숫자 배열로 해석하면 계산이 끝납니다. GPU에게는 사각형을 그리는 작업이었지만, 연구자에게는 배열 전체에 함수를 적용하는 작업이었던 것입니다.

## “All this just to do A + B”



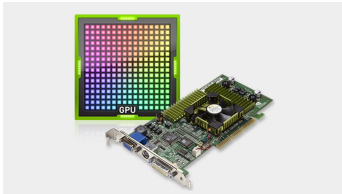
- The entry point still speaks graphics.
- Fragment programs have restricted output locations.
- Sharing data between executions is constrained.

## “All this just to do A + B”

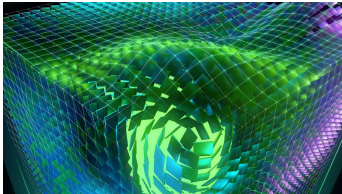
이 우스운 상황을 NVIDIA의 2006년 자료는 고작 A 더하기 B를 하려고 이 모든 것을 한다고 표현했습니다. 입력을 텍스처에 담고, 그리기 명령을 제출하고, 그래픽 파이프라인을 지나 결과를 색으로 저장하는 과정을 생각하면 정확한 표현입니다.

불편함은 코드가 길어진다는 데서 끝나지 않았습니다. 당시 프래그먼트 프로그램은 결과를 쓸 수 있는 위치가 제한됐고, 다른 실행과 데이터를 공유하는 데에도 제약이 있었습니다. 계산 능력은 쓸 수 있지만 입구부터 내부 규칙까지 그림을 그리는 장치에 맞춰져 있었던 것입니다. 그렇다면 이미 계산기로 쓰고 있는 사람들을 위해, 이 우회 경로를 정식 계산 인터페이스로 바꿀 수 있지 않을까요?

## CUDA made compute a first-class interface

**1999**

GeForce 256

**2006**

CUDA architecture

**launch()**

CUDA Toolkit 1.0

**2007**

CUDA Toolkit 1.0

**Allocate buffers. Run a kernel. Read and write data directly.**

Images / 2006: NVIDIA corporate timeline · Toolkit 1.0: NVIDIA guide, June 2007

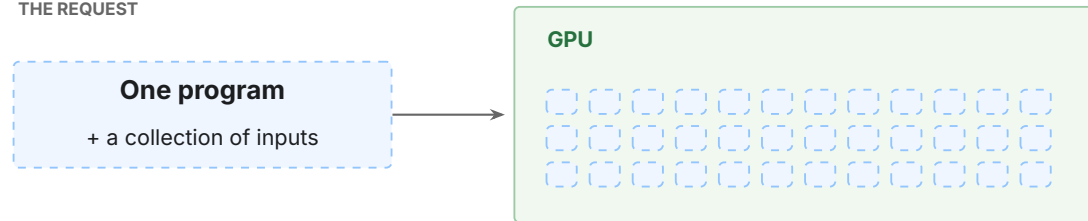
## CUDA made compute a first-class interface

NVIDIA는 GPU의 병렬 처리 능력을 과학과 연구용 계산에 직접 제공하는 방향으로 갔습니다. 2006년에 CUDA 아키텍처를 공개했고, 2007년에는 CUDA Toolkit 1.0이 나왔습니다. 그림의 2006년과 2007년은 각각 아키텍처 공개와 Toolkit 출시를 가리킵니다. 왼쪽의 그래픽 GPU에서 가운데의 계산 플랫폼으로 연결되는 변화라고 보면 됩니다.

핵심은 삼각형을 그리지 않고도 GPU의 프로그래밍 가능한 연산부에 일을 맡길 수 있게 됐다는 것입니다. GPU 메모리를 일반적인 데이터 버퍼로 다루고, 더 일반적인 읽기와 쓰기를 수행하고, 실행들 사이에서 데이터를 공유하는 기능을 제공했습니다. 단순히 셰이더의 이름만 바꾼 것이 아니라 계산을 시키는 입구와 가능한 동작이 함께 바뀐 것입니다.

## “Run this program many times.”

### THE REQUEST



The data-parallel structure survives.

Many logical executions

Now look inside the machine that accepts that request.

## “Run this program many times.”

사용자가 이제 하고 싶은 말은 이 프로그램을 많이 실행해달라는 것입니다. 화면을 설정하고 삼각형을 그리라는 말 대신, 실행할 프로그램과 데이터를 직접 건넵니다. 그래픽이라는 외피를 벗겨도 많은 데이터에 같은 계산을 적용한다는 성격은 그대로 남아 있습니다.

이제 연대기에서 잠시 벗어나겠습니다. 여기서부터는 실제 GPU가 역사적으로 이 순서대로 발명됐다는 이야기가 아니라, 이런 계산기를 설계한다면 어떤 구조가 필요할지 생각해 보는 관점입니다. 먼저 같은 일을 하는 실행들을 어떻게 묶는지 보고, 그 묶음을 실제로 처리할 하드웨어는 무엇을 갖춰야 하는지 살펴보겠습니다.

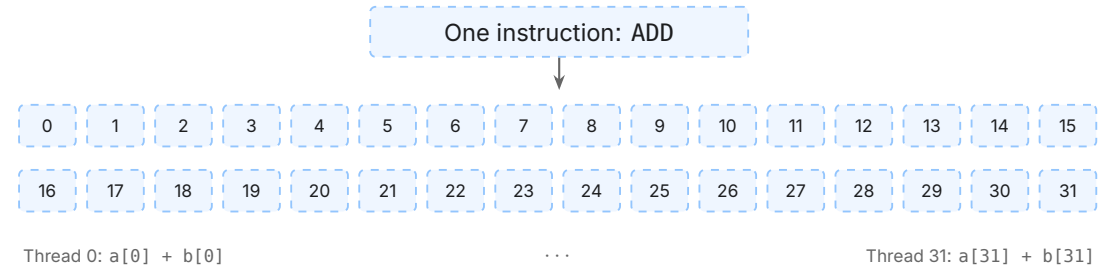
3. The execution engine

- Warps and SIMT
- SMs and GPU organization

The execution engine

다음은 GPU 안의 실행 엔진입니다. 같은 명령을 처리하는 thread를 warp로 묶는 이유를 보고, 그 실행에 필요한 상태와 자원을 SM 안에 모으는 설계 관점을 살펴보겠습니다. 여러 SM이 GPU로 확장되는 모습까지 연결합니다.

## A warp groups 32 logical threads



**SIMT** Single Instruction, Multiple Threads

**A warp describes grouped execution; it is not a 32-core component.**

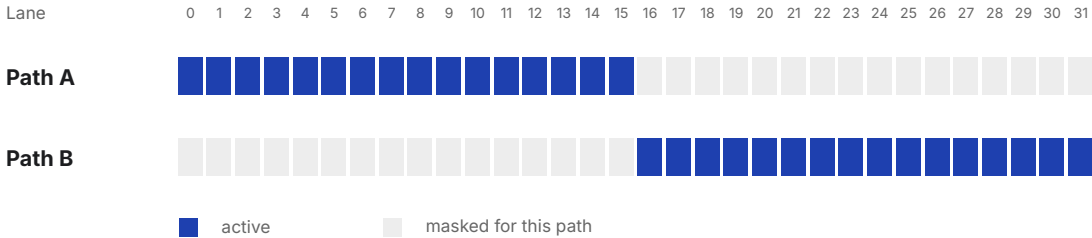
## A warp groups 32 logical threads

32개의 데이터에 대해 덧셈을 해야 한다고 해봅시다. 각 계산의 입력은 다르지만, 지금 해야 할 명령은 모두 덧셈입니다. 그렇다면 실행마다 따로 덧셈을 지시하기보다 하나의 명령으로 여러 데이터의 계산을 함께 진행하는 편이 효율적이겠죠.

NVIDIA는 프로그램의 개별 실행을 thread라고 하고, 같은 block 안의 thread를 32개씩 묶어 관리하는 단위를 warp라고 합니다. 프로그래머는 각 thread가 할 일을 쓰고, 하드웨어는 묶음 단위 실행을 활용합니다. 이것이 SIMT입니다. 여기서 파란 상자들은 논리적인 실행입니다. 32개의 물리적 연산기를 그린 것이 아닙니다. thread 하나가 특정 연산기를 영구적으로 하나씩 차지한다는 뜻도 아닙니다. 이 차이를 뒤에서도 계속 유지하겠습니다.

# Same program does not always mean the same path

```
if (lane < 16) { path_A(); } else { path_B(); }
```



Divergent paths are processed with different active threads.

NVIDIA CUDA Programming Guide · Warps and SIMT · conceptual mask illustration

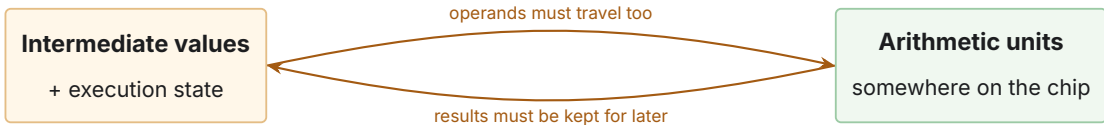
# Same program does not always mean the same path

같은 코드를 실행해도 모든 thread가 같은 분기로 들어가는 것은 아닙니다. 여기서는 앞의 16개 thread는 A 경로로, 나머지 16개는 B 경로로 간다고 했습니다. A 경로의 명령을 처리할 때에는 그 경로에 참여하지 않는 thread를 비활성화합니다. B 경로를 처리할 때에는 반대쪽 thread가 참여합니다.

이것이 warp divergence입니다. 서로 다른 경로의 작업을 처리해야 하므로 같은 warp 안에서 실행 경로가 비슷할수록 자원을 효율적으로 활용하기 쉽습니다. 그림은 활성 마스크를 보여주는 개념도이고 정확한 사이클표는 아닙니다. 최신 구조의 독립적인 thread 스케줄링 같은 세부 사항과 무관하게, warp를 이해할 때 모든 thread가 언제나 한 경로를 똑같이 간다고 생각하면 안 된다는 점을 기억하면 됩니다.

## Arithmetic needs state, and state has to live somewhere

A DESIGN THOUGHT EXPERIMENT



Store values.      Track progress.      Choose the next instruction.

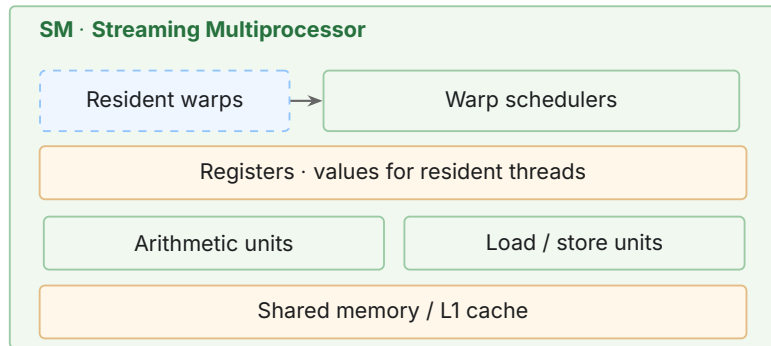
**Adding execution units also creates a data-movement problem.**

## Arithmetic needs state, and state has to live somewhere

그러면 warp를 처리할 연산기를 칩 안에 잔뜩 넣으면 끝일까요? warp는 일의 묶음이므로 실제로 계산하는 부품은 따로 필요합니다. 게다가 프로그램은 덧셈 한 번으로 끝나지 않습니다. 값을 읽고, 곱하고, 더하고, 조건을 검사하려면 중간값과 실행 상태를 기억할 자리, 다음 명령을 고르는 장치가 있어야 합니다.

설계 관점에서 모든 warp를 한곳에서 관리하고 칩 전체의 빈 연산기로 보내는 구조를 상상해봅시다. 값은 왼쪽에 있고 빈 연산기는 오른쪽에 있다면 명령만 보내서는 안 됩니다. 필요한 값도 이동해야 합니다. 결과를 다시 가져오거나 멀리서 접근할 연결망도 필요합니다. 이것은 실제 설계의 역사를 주장하는 그림이 아니라, 연산기를 늘리면 상태의 저장과 이동도 함께 고민해야 한다는 직관을 보여 줍니다.

## An SM keeps execution and its working state close



**Manage many warps.**

Keep their state nearby.

Feed ready instructions to execution units.

Conceptual organization.  
Internal partitions vary by architecture.

**Warp = grouped work. SM = the hardware that executes it.**

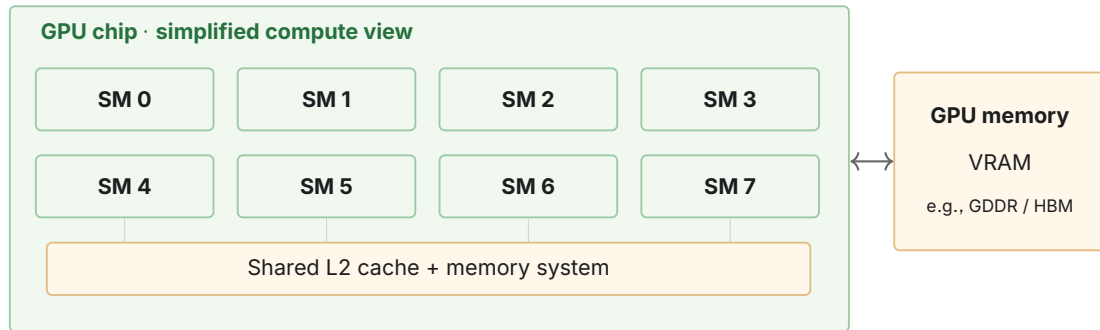
## An SM keeps execution and its working state close

그렇다면 서로 자주 주고받는 것들을 가까이 묶어봅시다. 연산기 옆에 중간값을 보관할 레지스터를 두고, 실행을 관리할 스케줄러와 가까워서 쓸 메모리도 함께 둡니다. NVIDIA GPU에서 이런 역할을 하는 단위가 SM, Streaming Multiprocessor입니다.

SM은 연산 유닛, 레지스터, warp 스케줄러, 메모리 읽기와 쓰기를 담당하는 장치, shared memory와 L1캐시 같은 자원을 갖습니다. SM 하나가 warp 하나 전용인 것은 아닙니다. 여러 warp의 실행 상태를 보관하고, 그중 준비된 명령을 골라 자원에 공급합니다. 보관하는 warp 수와 순간적으로 명령을 처리하는 연산기 수는 다른 숫자입니다. 실제 SM에는 스케줄러가 여러 개일 수 있고 내부가 여러 구획으로 나뉘기도 합니다. 이 그림은 그 세부 구조를 묶어 본 실행 엔진입니다.

NVIDIA CUDA Programming Guide · GPU Hardware Model · original schematic

## Scale the GPU by repeating that execution engine



**Share control within a warp; keep execution resources local to an SM.**

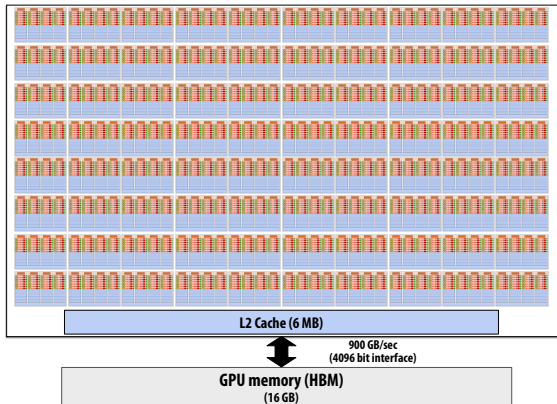
NVIDIA CUDA Programming Guide · GPU Hardware Model · illustrative SM count

## Scale the GPU by repeating that execution engine

이제 GPU를 크게 만드는 방법도 달라집니다. 하나의 실행 엔진을 끝없이 키우기보다, 자기 몫의 실행을 관리할 수 있는 SM을 여러 개 배치합니다. 각 SM은 가까운 레지스터와 실행 자원을 사용하고, 바깥의 공유 캐시와 메모리 시스템을 통해 데이터에 접근합니다.

실제 GPU에는 이 그림보다 많은 구성 요소와 중간 계층이 있지만, 범용 계산의 중심을 보면 여러 SM으로 이루어진 구조입니다. warp에서는 같은 명령을 함께 처리해서 제어 비용을 나누고, SM에서는 자주 주고받는 상태와 연산 자원을 가까이 묶습니다. 공유해서 효율을 얻으면서 공유 범위를 나눠 확장하는 설계 관점입니다. 오른쪽의 VRAM은 텐서 같은 데이터를 담는 GPU 메모리입니다. SM의 레지스터나 shared memory와 같은 저장 공간은 아닙니다.

## A concrete example: the NVIDIA V100



### 80 SMs

Repeated execution engines

### Shared L2 cache

A cache shared by the SMs

### HBM2 memory

Stores the inputs and outputs

Historical example, not a universal GPU specification.

Stanford CS149, Fall 2025 · slide 52 · V100 configuration shown in the lecture

## A concrete example: the NVIDIA V100

실제 사례를 하나 보겠습니다. Stanford 강의에서 설명하는 NVIDIA V100 구성에는 SM이 80개 있습니다. 앞에서 그린 SM 상자를 실제 칩에서는 여러 번 반복하고 있다는 점을 보시면 됩니다. 각 SM 안에는 다시 스케줄러, 레지스터, 연산 유닛과 가까운 메모리가 있습니다.

아래에는 GPU 안에서 공유하는 L2 캐시가 있고, 그 바깥에는 큰 데이터를 저장하는 HBM2 메모리가 연결됩니다. 여기서는 세부 사양 숫자를 외울 필요가 없습니다. 작은 실행 엔진 여러 개와 공유 메모리 시스템이라는 큰 그림이 실제 사례에서도 유지된다는 것을 확인하면 됩니다. V100이라는 특정 세대의 예시이므로, 모든 NVIDIA GPU의 SM 개수나 구성이라고 받아들이면 안 됩니다.

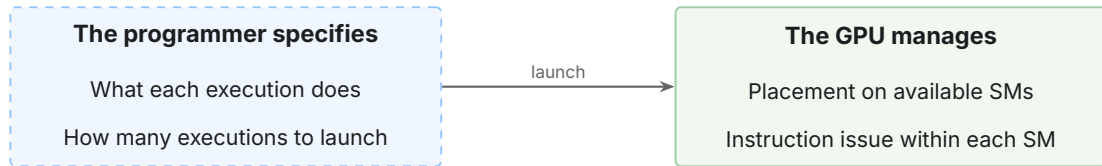
## 4. Following A + B

- From tensors to threads
- Scheduling and memory
- Utilization and completion

## Following A + B

마지막으로 PyTorch의 C는 A 더하기 B라는 한 줄을 따라가겠습니다. 입력 데이터의 위치, thread와 block의 구성, SM 배치와 warp 명령 실행, 캐시와 레지스터 접근을 연결한 다음 결과가 언제 완성되는지 정리하겠습니다.

## Describe the work; the GPU places it



PyTorch chooses CUDA implementations underneath tensor operations.

**A work description is not a physical placement plan.**

## Describe the work; the GPU places it

여기까지는 아래에서 위로 기계를 만들었습니다. 이제 사용자의 입장으로 돌아오면, SM이 여러 개 있으니 내가 SM 0에는 이 일을, SM 1에는 저 일을 직접 배정해야 하는지 궁금해집니다. CUDA는 보통 그렇게 사용하지 않습니다.

프로그래머는 각 실행이 무엇을 할지, 그리고 그 실행들을 어떻게 묶어서 얼마나 실행할지 표현합니다. 구체적으로 어느 물리적 연산기를 사용할지 지시하는 배치도를 쓰는 것은 아닙니다. GPU가 사용 가능한 자원에 맞춰 일을 배치하고, 각 SM이 명령을 실행합니다. PyTorch에서는 이런 실행 설정도 더 아래에 숨겨져 있습니다. 이제 텐서에 대한 한 줄의 요청에서 출발해 방금 본 SM과 warp까지 내려가보겠습니다.

## Start with one line of PyTorch

```
import torch
A = torch.randn(1024, 1024, device="cuda")
B = torch.randn(1024, 1024, device="cuda")
C = A + B
```

A



B



+

=

C



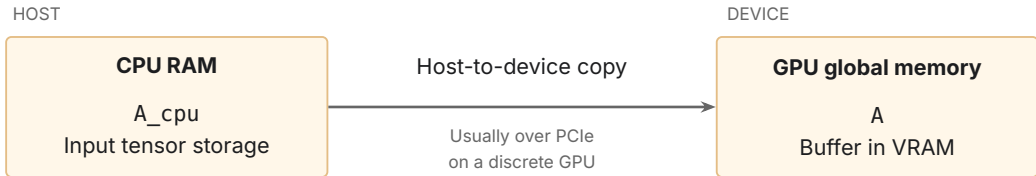
**Each output needs only its own two inputs. No neighbor must finish first.**

## Start with one line of PyTorch

GPU에 있는 1024 곱하기 1024 행렬 두 개를 더하는 코드입니다. C는 A와 B의 같은 위치에 있는 원소끼리 더한 결과입니다. 행렬 전체를 하나의 숫자로 합치는 연산이 아니라, 같은 크기의 행렬을 만드는 원소별 덧셈입니다.

여기서 첫 번째 덧셈이 끝나야 두 번째 덧셈을 할 수 있을까요? 그렇지 않습니다. 각 결과에 필요한 것은 자기 위치의 입력 두 개뿐이고, 옆 원소의 결과를 기다릴 필요가 없습니다. 따라서 백만 개가 넘는 덧셈을 여러 실행에 나눠 맡길 수 있습니다. 많은 데이터가 있고, 같은 계산을 적용할 수 있고, 서로 다른 데이터의 계산이 독립적이라는 처음의 구조가 다시 나타났습니다. 다음부터는 이해를 위해 thread 하나가 원소 하나를 처리하는 교육용 CUDA 구현을 따라가겠습니다. 실제 PyTorch의 최적화된 실행 설정과 동일하다는 뜻은 아닙니다.

## If the input starts in CPU memory, copy it to the GPU



```
A_cpu = torch.randn(1024, 1024, device="cpu")
A = A_cpu.to("cuda")
```

**With `device="cuda"`, our original example already creates GPU tensors.**

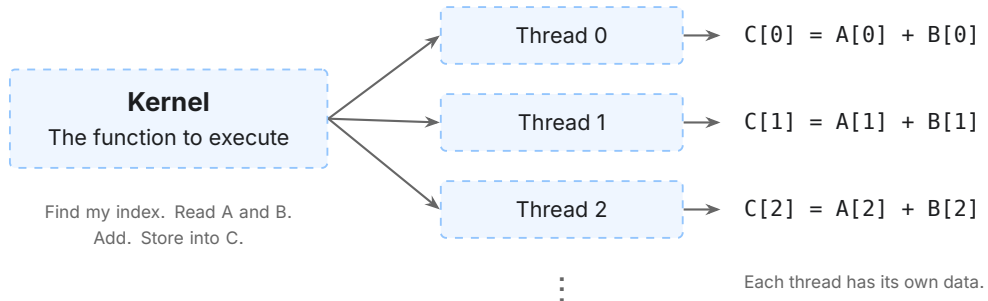
PyTorch · CUDA semantics and device transfers

## If the input starts in CPU memory, copy it to the GPU

계산에 들어가기 전에 입력 데이터가 어디서 출발했는지를 잠깐 짚겠습니다. 만약 데이터를 CPU에서 만들거나 CPU 메모리로 읽었다면, 처음에는 왼쪽의 RAM에 있습니다. 아래 첫 줄은 device에 CPU를 지정해서 그런 출발점을 보여줍니다. 다음 줄의 `to cuda`는 그 값을 GPU 쪽 저장 공간으로 옮깁니다. 일반적인 별도 그래픽카드에서는 이 복사가 PCIe 연결을 지나고, 도착한 데이터는 VRAM에 있는 GPU의 global memory 버퍼에 놓입니다.

하지만 방금 본 원래 코드에는 `device cuda`가 있었습니다. 그 경우에는 처음부터 GPU 텐서를 생성하므로, 이 CPU 입력 복사 단계를 따로 거칠 필요가 없습니다. 지금 그림은 CPU에 있던 입력을 사용하는 경우에만 추가되는 경로입니다. 또 복사의 도착점은 global memory입니다. shared memory나 warp별 메모리를 차례로 통과하는 경로가 아닙니다. 실제 커널이 이 입력을 사용할 때 SM 내부의 자원으로 어떻게 읽어오는지는 뒤에서 실행 명령과 함께 이어서 보겠습니다.

## One program, many logical executions



**A kernel is code. A thread is one logical execution of that code.**

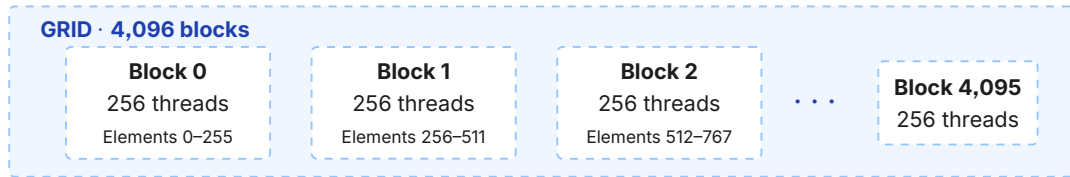
CUDA Programming Guide · Kernel and thread model

## One program, many logical executions

이제 행렬 덧셈이라는 요청을 GPU가 실행할 형태로 표현해보겠습니다. GPU에서 실행할 함수를 커널이라고 부릅니다. 그리고 그 함수를 실행하는 논리적인 인스턴스 하나가 thread입니다. 커널에는 자기 원소 번호를 알아내고, A와 B의 값을 읽고, 더해서 C의 같은 위치에 저장하라는 내용을 씁니다. 왼쪽에는 프로그램이 하나만 있습니다. 오른쪽의 thread들은 모두 그 프로그램을 실행하지만, 맡은 데이터의 번호가 다릅니다. 앞에서 warp를 설명할 때 개별 실행이라고 했던 것이 바로 이 thread입니다. 서로 다른 프로그램을 백만 개 만드는 것이 아닙니다. 여기서는 이해를 위해 thread 하나가 원소 하나를 처리하는 교육용 구현을 따라가겠습니다. 실제 PyTorch 구현은 thread 하나에 여러 원소를 맡기는 등의 최적화를 사용할 수 있습니다. 따라서 이후에 나오는 실행 설정은 실제 PyTorch 내부의 고정된 설정이 아니라, 요청이 어떤 실행 구조로 내려가는지를 보여주는 예입니다.

## How many executions do we launch?

$1024 \times 1024 = \mathbf{1,048,576}$  elements



```
add<<<4096, 256>>>(device_a, device_b, device_c, 1024 * 1024);
```

**4,096 blocks × 256 threads = 1,048,576 logical threads.**

## How many executions do we launch?

우리 행렬은 1024 곱하기 1024이므로 출력 원소가 104만 8576개입니다. thread 하나에 원소 하나를 맡긴다면 논리적인 실행도 그만큼 필요합니다. CUDA는 thread를 block으로 묶고, 이번 커널 실행에 참여하는 block 전체를 grid라고 부릅니다. 여기서는 설명용으로 block 하나에 thread 256개를 넣겠습니다. 그러면 첫 block은 0번부터 255번까지, 다음 block은 256번부터 511번까지를 처리합니다. 전체 원소 수를 256으로 나누면 필요한 block은 4096개입니다. 아래 코드의 꺾쇠 안에는 그 두 숫자가 들어갑니다. add라는 커널을 block 4096개로 실행하되 각 block에는 thread 256개를 넣으라는 뜻입니다. 이렇게 실행할 프로그램과 실행 규모를 지정해서 제출하는 것이 kernel launch입니다. 아직 어느 SM에 어떤 block이 올라갈지는 쓰지 않았습니다. 지금 정한 것은 하드웨어 배치가 아니라 전체 작업의 구성입니다.

## Every thread can find its own element

```
__global__ void add(
    const float* a,
    const float* b,
    float* c,
    int element_count
) {
    int index = blockIdx.x * blockDim.x
               + threadIdx.x;
    if (index < element_count) {
        c[index] = a[index] + b[index];
    }
}
```

Same code, different built-in IDs, different data.

BLOCK 3, THREAD 10

$$\underbrace{3 \times 256}_{\text{block offset}} + \underbrace{10}_{\text{local index}}$$

778

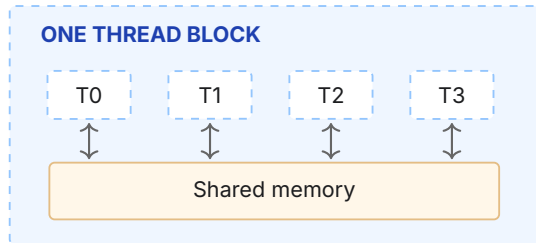
$C[778] = A[778] + B[778]$

The bounds check protects the final block when the size is not divisible by 256.

## Every thread can find its own element

모든 thread가 같은 코드를 실행하는데 각자 맡은 원소는 어떻게 구분할까요? CUDA가 제공하는 번호를 이용하면 됩니다. blockIdx는 자기 block의 번호이고, blockDim은 한 block의 크기이며, threadIdx는 block 안에서 자기 번호입니다. block 번호에 block 크기를 곱하면 그 block이 시작하는 원소 번호가 됩니다. 여기에 자기 thread 번호를 더합니다. 오른쪽 예에서는 block 3의 thread 10이므로 3 곱하기 256에 10을 더해서 778번 원소를 말합니다. 이 실행은 A의 778번과 B의 778번을 읽어서 C의 778번에 저장합니다. 코드의 if문도 봐주세요. 이번 크기는 256으로 정확하게 나누어떨어지지만 항상 그런 것은 아닙니다. 마지막 block에 실제 원소보다 많은 thread가 생길 수 있어서 배열 범위를 확인합니다. 행렬을 일렬로 번호 매기는 것은 이번 교육용 예제를 단순하게 설명하기 위한 선택입니다. 핵심은 각 실행이 자신의 논리적인 위치를 바탕으로 담당 데이터를 결정한다는 점입니다.

## A block is also a cooperation boundary



### Share data locally

Threads in a block can exchange data.

### Synchronize within the block

`__syncthreads()` provides a block-wide barrier.

**256 is our choice.** A common device limit is 1,024 threads per block; kernel resources can constrain it further.

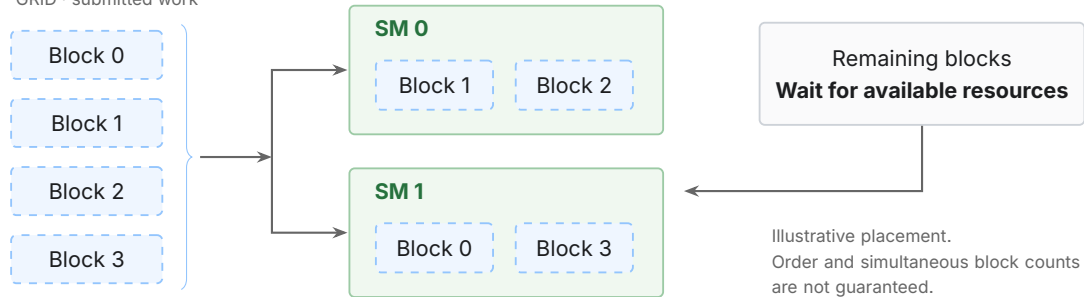
**This addition needs no shared intermediate values or block barrier.**

## A block is also a cooperation boundary

block은 단순히 원소 번호를 계산하기 위한 포장만은 아닙니다. 같은 block의 thread들은 shared memory라는 가까운 저장 공간으로 데이터를 주고받을 수 있습니다. 또 `syncthreads`를 이용해 block 안의 thread들이 특정 지점까지 도착하도록 동기화할 수 있습니다. 예를 들어 여러 실행의 중간 결과를 합치는 계산에서는 이런 기능이 필요할 수 있습니다. 하지만 지금의 원소별 덧셈은 자기 입력 두 개만 있으면 됩니다. 옆 thread의 결과를 기다리지 않으므로 협력 기능을 사용하지 않습니다. block 크기에 대해서도 하나 짚고 넘어가겠습니다. 256은 하드웨어에 고정된 숫자가 아니라 우리가 고른 값입니다. 물론 무한히 크게 만들 수는 없습니다. 대표적인 장치 상한은 block당 1024 thread이고, 커널이 쓰는 레지스터 등의 자원도 제약이 됩니다. 1024라는 수는 block 하나에 대한 제한이지 GPU 전체가 실행할 수 있는 thread 수의 제한은 아닙니다. 이제 이렇게 구성한 block이 실제 기계에 배치되는 모습을 보겠습니다.

## Many blocks share a finite set of SMs

GRID · submitted work

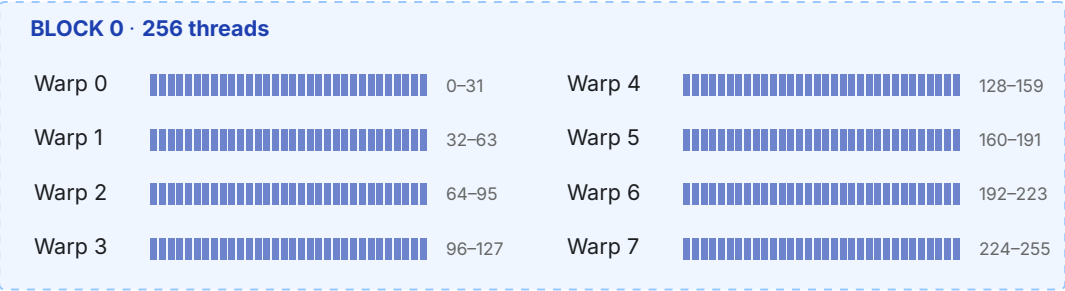


**One block runs on one SM. An SM may host several blocks.**

## Many blocks share a finite set of SMs

백만 개의 thread를 실행하려면 물리적인 코어도 백만 개가 필요할까요? 그렇지 않습니다. 앞에서 살펴본 SM이 여기서 실제 실행 엔진으로 다시 등장합니다. GPU는 제출한 grid의 block들을 SM에 배치합니다. 그림처럼 번호 순서대로 대응할 필요는 없으며 구체적인 순서도 보장되지 않습니다. 중요한 규칙은 block 하나가 같은 SM에서 실행된다는 점입니다. 반대로 SM 하나에는 레지스터, shared memory, 실행 상태를 보관할 공간 등이 허용하는 만큼 여러 block이 함께 올라갈 수 있습니다. 모든 block이 한꺼번에 올라갈 필요도 없습니다. 자원이 차면 남은 block은 기다리고, 실행 중인 block이 끝나서 자리가 나면 다음 block이 배치됩니다. 따라서 그림의 block 개수나 배치는 설명을 위한 예시입니다. 실제 수용량은 GPU와 커널에 따라 달라집니다. grid는 이번에 제출한 작업 전체이고 GPU는 그것을 나누어 처리하는 기계입니다. block이 4096개라고 SM도 4096개 필요하지는 않습니다.

## A 256-thread block contains eight warps

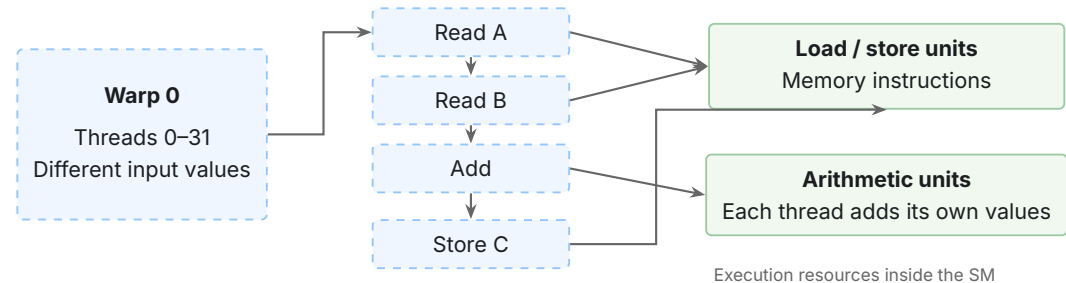


Within a block, consecutive groups of 32 threads form warps.

## A 256-thread block contains eight warps

SM에 올라간 block 하나를 확대해보겠습니다. 이번 block에는 thread가 256개 있습니다. NVIDIA GPU는 같은 block 안의 thread들을 연속된 번호 순서로 32개씩 묶어 warp로 관리합니다. 따라서 여기서는 warp가 여덟 개 만들어집니다. 첫 warp는 thread 0번부터 31번이고, 다음 warp는 32번부터 63번입니다. 이런 식으로 마지막 warp는 224번부터 255번을 말합니다. 그림의 작은 막대는 물리적인 코어가 아니라 논리적인 thread입니다. warp는 앞에서 보았듯 일을 묶어 명령을 처리하는 단위이기 때문입니다. thread는 프로그래머가 작성한 커널의 개별 실행이고, block은 그 실행들을 협력 가능하게 묶은 단위이며, warp는 SM이 실행을 관리할 때 사용하는 묶음입니다. 이 세 개는 서로 경쟁하는 이름이 아니라 서로 다른 관점에서 같은 작업을 설명하는 이름입니다. 이제 여덟 warp 가운데 첫 번째 warp가 덧셈을 수행하는 과정을 더 가까이서 보겠습니다.

## A warp needs different units for different instructions

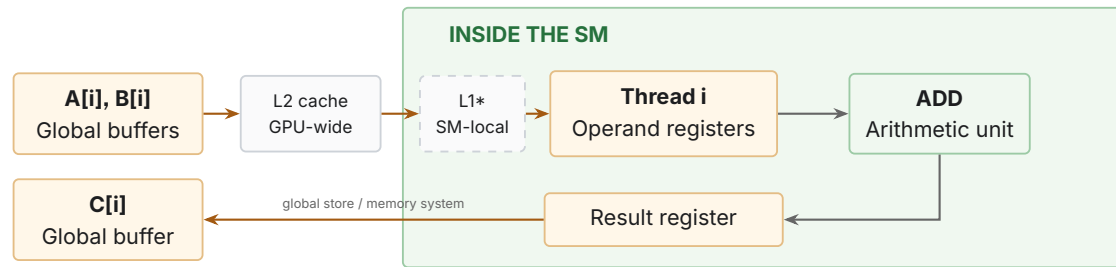


**The instruction is shared; the data belongs to each thread.**

## A warp needs different units for different instructions

warp 0의 thread들이 해야 할 일은 같습니다. 자기 A 값을 읽고, 자기 B 값을 읽고, 둘을 더하고, 자기 C 위치에 저장합니다. 덧셈 명령이 발행되면 0번 thread는 A의 0번과 B의 0번을 더하고, 31번 thread는 각 배열의 31번을 더합니다. 명령은 함께 처리하지만 데이터는 서로 다릅니다. 이것이 우리의 행렬 덧셈에서 나타나는 SIMT입니다. 또 네 단계를 모두 같은 종류의 연산기가 처리하는 것은 아닙니다. 메모리 읽기와 쓰기는 load/store 경로를 이용하고, 덧셈은 산술 연산 유닛을 사용합니다. 실제로는 주소 계산 같은 명령도 필요하지만, 여기서는 데이터가 읽히고 계산되고 저장되는 흐름을 보여주기 위해 단순화했습니다. warp가 특정 연산기를 붙잡고 프로그램이 끝날 때까지 소유하는 것이 아닙니다. 명령을 실행할 때 그 종류에 맞는 SM의 자원을 사용합니다. 이 점을 기억하면 다음 슬라이드에서 왜 여러 warp가 번갈아 진행될 수 있는지도 자연스럽게 이어집니다.

## For this addition, operands are loaded into thread registers



\*Caches may serve the load; L1 use and the load/store path depend on architecture and policy.

**The add kernel uses per-thread registers. Shared-memory staging is unnecessary.**

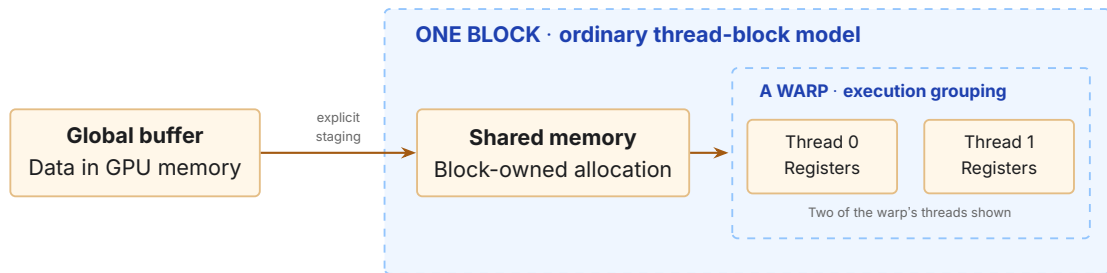
NVIDIA PTX ISA · Cache operators · logical data flow, not a physical floorplan

## For this addition, operands are loaded into thread registers

이제 A와 B가 GPU 메모리에 있다고 합시다. 실제 ADD 명령이 계산할 값은 어디에 있을까요? thread는 자기 원소를 읽는 global load 명령으로 값을 가져와 작업용 레지스터에 둡니다. 그 값을 산술 연산 유닛이 더하고, 결과도 레지스터에 만들어진 다음 global store로 C의 위치에 저장됩니다. 레지스터의 값은 논리적으로 각 thread에 속하며, 물리적으로는 SM의 레지스터 파일에 보관됩니다.

읽는 과정에서 캐시가 요청을 처리할 수 있습니다. 이미 값이 캐시에 있으면 매번 VRAM까지 갈 필요가 없고, L1을 사용하는지도 아키텍처와 명령의 정책에 따라 달라집니다. L2는 GPU의 여러 SM이 공유하는 캐시이고, L1은 그림처럼 각 SM 안에 있습니다. 이 그림은 가능한 접근 경로를 요약하며, 모든 읽기가 항상 각 단계를 거친다고 가정하지 않습니다. 쓰기의 캐시 정책도 읽기와 같다고 가정하지 않습니다. 이번 덧셈은 각 원소를 읽어 한 번 더하면 되므로 shared memory로 따로 옮기는 단계가 필요 없습니다.

## Shared memory is explicit storage for block cooperation



Useful when threads reuse or exchange data. This addition does not take this optional path.

**There is no separate “warp memory” tier between shared memory and registers.**

NVIDIA CUDA Programming Guide · On-chip memory, thread blocks, and warps

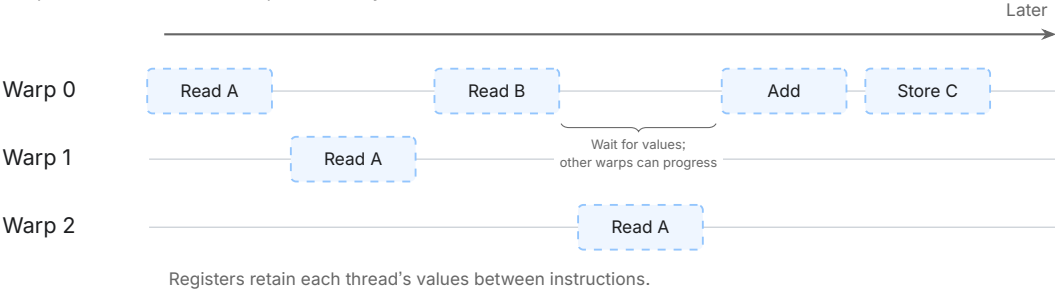
## Shared memory is explicit storage for block cooperation

그렇다면 shared memory는 언제 쓰는 것일까요? 같은 block의 thread들이 데이터를 함께 재사용하거나 중간 결과를 주고받고 싶을 때, 커널이 명시적으로 shared memory에 데이터를 준비할 수 있습니다. 앞에서 본 캐시는 하드웨어가 관리하지만, shared memory의 내용과 사용 순서는 프로그램이 관리합니다. 데이터를 준비한 뒤에는 각 thread가 필요한 값을 자기 레지스터로 읽어서 계산합니다. 여기서는 이런 협력의 논리적인 흐름을 그렸고, 실제 복사 경로는 명령과 아키텍처에 따라 달라질 수 있습니다.

따라서 CPU 메모리, GPU global memory, shared memory, warp memory, register가 모두 일렬로 이어지는 필수 단계는 아닙니다. 특히 warp 전용 메모리 계층은 없습니다. warp는 실행을 묶는 단위이고, 레지스터 값은 각각의 thread에 속합니다. shared memory는 기본 block 모델에서 block이 함께 쓰는 공간입니다. L1과 shared memory가 같은 물리 저장 자원을 나누어 쓰는 GPU도 있지만, 관리 방식과 프로그램에서의 의미는 서로 다릅니다.

# The scheduler issues the next ready instruction

One possible issue order · conceptual, not a cycle-accurate trace



**A warp does not have to finish its whole program before another runs.**

# The scheduler issues the next ready instruction

스케줄러가 warp 하나의 프로그램을 처음부터 끝까지 실행한 다음 다음 warp로 넘어가는 것은 아닙니다. 준비된 warp의 다음 명령을 골라 발행합니다. 그림은 그 흐름을 보여주는 가능한 예시이고 정확한 사이클표나 실제 어셈블리 순서는 아닙니다. warp 0이 A를 읽는 명령을 내보낸 다음 warp 1도 A를 읽을 수 있습니다. 그 뒤 warp 0의 B 읽기와 warp 2의 A 읽기가 이어질 수 있습니다. 읽어온 값이 준비되면 warp 0은 덧셈을 실행하고 나중에 C에 저장합니다. 어떤 warp가 메모리에서 값이 오기를 기다리는 동안 다른 warp의 준비된 명령을 진행시킬 수 있다는 점이 중요합니다. 각 실행의 중간값은 레지스터 등의 상태에 남아 있으므로 명령을 바꿔 발행한다고 값이 사라지지 않습니다. 이것이 SM 안에 연산기뿐 아니라 실행 상태와 스케줄러가 함께 필요했던 이유입니다. 전체로 보면 행렬 덧셈이지만 SM에서는 이런 읽기, 산술, 쓰기 명령들이 여러 warp에 걸쳐 진행됩니다.

# Warp width and execution width are different numbers

## VOLTA / V100 EXAMPLE



A 32-thread FP32 arithmetic instruction is processed over **two cycles**.

V100 has four warp schedulers / partitions per SM. Other instructions and architectures differ.

**32 threads does not imply 32 dedicated units finishing in one clock.**

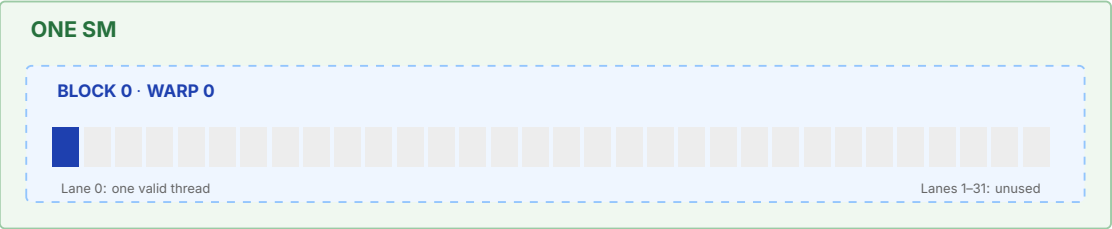
# Warp width and execution width are different numbers

여기서 32라는 숫자를 한 번 더 구분하겠습니다. warp의 폭은 thread 32개이지만, 이것이 물리적인 연산기 32개가 반드시 한 클럭에 끝낸다는 뜻은 아닙니다. V100의 Volta 구조를 예로 들면, SM 안에는 네 개의 스케줄러 구획이 있고 각 구획은 전용 실행 자원을 갖습니다. 그중 FP32 산술 연산기는 구획당 16개입니다.

따라서 이 구성에서는 32개 thread의 FP32 산술 명령을 두 사이클에 걸쳐 처리합니다. 이것은 명령의 결과가 두 사이클 뒤에 반드시 준비된다는 지연 시간 설명과도 구분해야 합니다. 그림은 warp의 논리적인 폭과 실제 실행 폭이 다를 수 있다는 사례입니다. 모든 GPU나 모든 명령이 같은 구성을 갖는다는 뜻은 아닙니다. 핵심은 thread 수, 연산기 수, 명령 처리 시간을 서로 다른 개념으로 보자는 것입니다.

# What if we launch just one thread?

```
add<<<1, 1>>>(device_a, device_b, device_c, 1);
```



One active thread in a warp on one SM.

NVIDIA CUDA Programming Guide · Partial warps · assumes no other GPU work

# What if we launch just one thread?

반대로 GPU에 다른 작업이 없는 상태에서 thread 하나만 실행하면 어떻게 될까요? block 하나에 thread 하나를 넣어 실행해봅시다. 이때도 실행을 관리할 warp가 생깁니다. 다만 그 warp에서 실제 작업을 가진 thread는 하나뿐이고 나머지 lane은 사용되지 않습니다.

이 block은 하나의 SM에 배치됩니다. 따라서 CUDA core 하나를 프로그램이 끝날 때까지 영구적으로 점유한다고 설명하기보다는, 한 SM에서 실행되는 warp에 유효한 thread가 하나뿐이라고 말하는 것이 정확합니다. 그 thread도 값을 읽을 때에는 메모리 경로를 쓰고, 덧셈을 할 때에는 산술 자원을 씁니다. 지금까지 구분했던 논리적인 실행, 묶음, 물리적인 실행 자원을 적용하면 작은 요청도 같은 구조로 설명할 수 있습니다.

# More threads in one block still means one SM

## ONE LARGE BLOCK

add<<<1, 1024>>>(...)



If supported by the device and kernel

## MANY BLOCKS

add<<<4096, 256>>>(...)



Blocks available for multiple SMs

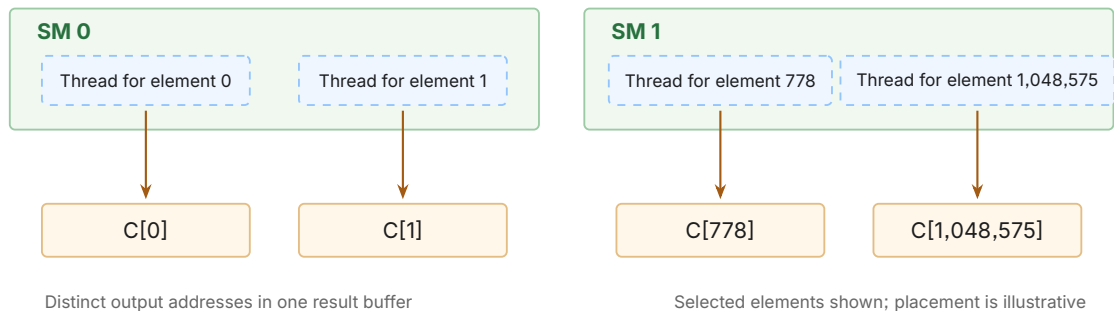
Fill the warps, and provide blocks that can spread across SMs.

# More threads in one block still means one SM

이번에는 block 하나를 1024개 thread로 키워봅시다. 이 설정이 허용되는 장치와 커널이라면 thread가 1024개이므로 warp는 32개입니다. 하지만 block이 하나라는 사실은 바뀌지 않습니다. 그 block은 여전히 하나의 SM에서 실행되고, thread가 많아졌다고 여러 SM으로 쪼개지지 않습니다.

오른쪽은 원래 사용하던 4096개 block, block당 256개 thread 설정입니다. 이 경우에는 여러 SM에 나눠 맡길 block이 충분히 있습니다. 그림의 SM 네 개는 원리를 보여주는 예시이고, 배치 순서나 각 SM의 동시 수용량을 약속하는 것은 아닙니다. thread 수가 많다는 말과 GPU 전체에 일이 잘 분산된다는 말이 다른 이유가 여기에 있습니다. GPU를 활용하려면 warp 안에 할 일이 있어야 하고, 여러 SM에 올릴 block도 있어야 합니다.

## The result is already in C, in GPU memory



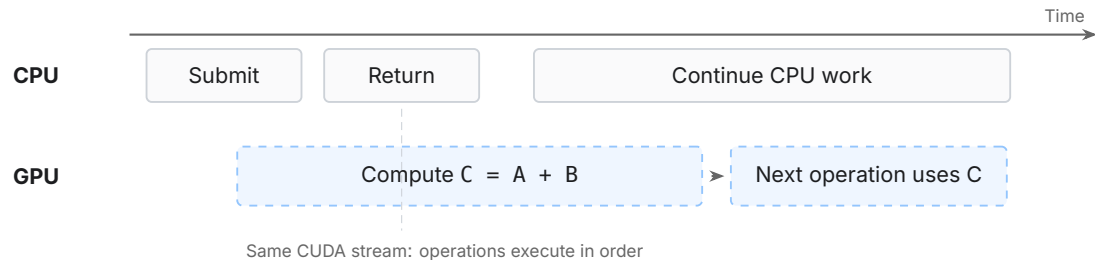
**Each thread stores its own result. The CPU does not reassemble the matrix.**

## The result is already in C, in GPU memory

우리 커널의 각 thread는 자기 원소를 계산한 다음 GPU 메모리의 C에 직접 씁니다. 서로 다른 thread가 서로 다른 출력 위치를 맡으므로, CPU가 각 SM에서 계산한 조각을 받아서 다시 행렬로 조립할 필요는 없습니다. 그림에서는 몇 개의 원소만 골라 보여주고 있고, 실제 배치 순서는 이 그림처럼 정해져 있는 것은 아닙니다.

모든 작업이 완료되면 결과는 이미 C의 각 위치에 들어 있습니다. 이것이 프로그램 전체에서는 행렬 덧셈으로 보이지만, 개별 thread에게는 자기 원소의 덧셈이라는 설명이 연결되는 지점입니다. 다만 여기서 완료됐다는 말은 조심해서 써야 합니다. CPU 쪽의 함수 호출이 반환됐다는 사건과 GPU가 이 저장 작업을 모두 끝냈다는 사건은 서로 다른 시점일 수 있습니다.

## Python can return before the GPU finishes



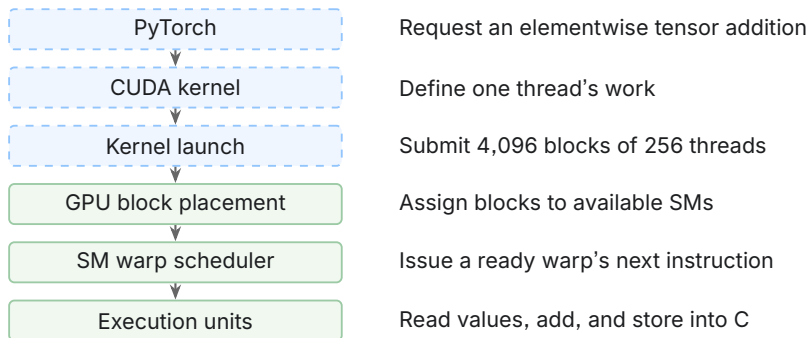
- GPU operations are normally submitted asynchronously.
- Reading results on the CPU or measuring GPU time needs appropriate synchronization.

## Python can return before the GPU finishes

PyTorch의 GPU 연산은 기본적으로 비동기적으로 제출됩니다. CPU는 작업을 제출한 뒤 함수에서 돌아와 다음 일을 진행할 수 있고, GPU에서는 덧셈이 아직 진행 중일 수 있습니다. 따라서 C는 A 더하기 B라는 코드가 반환됐다고 해서 그 순간 GPU의 모든 계산이 끝났다고 해석하면 안 됩니다.

그렇다고 다음 GPU 연산이 C를 사용할 때마다 CPU가 직접 기다려야 하는 것도 아닙니다. 같은 CUDA stream에 제출한 작업은 순서가 유지되므로 C를 사용하는 다음 작업은 앞선 덧셈 뒤에 실행됩니다. 반면 CPU에서 결과를 읽거나 실제 GPU 계산 시간을 재려면 필요한 동기화가 있어야 합니다. CPU에서 호출 전후 시간만 재면 제출 비용만 측정할 수 있기 때문입니다. 이 그림은 같은 stream의 순서만 보여주며, 서로 다른 stream 사이의 의존성은 별도로 관리해야 합니다.

## Follow the request all the way down



**One tensor operation becomes many scheduled instructions on real hardware.**

## Follow the request all the way down

이제 처음부터 끝까지 연결해보겠습니다. PyTorch에서는 C는 A 더하기 B라는 텐서 연산을 요청합니다. 아래에는 각 실행이 값을 읽고 더해서 저장하는 CUDA 커널 구현이 있습니다. 이번 교육용 구현에서는 block 4096개와 block당 thread 256개로 실행 규모를 표현했습니다.

GPU는 제출된 block을 자원이 있는 SM에 배치합니다. SM 안에서는 warp 스케줄러가 준비된 warp의 다음 명령을 골라 발행합니다. 명령의 종류에 따라 실제 연산 유닛과 메모리 경로가 값을 읽고, 더하고, C에 저장합니다. 사용자는 행렬 덧셈 하나를 보지만, 아래에서는 수많은 실행이 묶이고 나뉘어 실제 하드웨어의 명령으로 진행됩니다. 위쪽 파란 영역은 표현한 작업이고 아래쪽 초록 영역은 그것을 처리하는 기계라는 구분도 다시 보시면 좋겠습니다.

# Keep the work and the machine separate

|        |                                                  |
|--------|--------------------------------------------------|
| Kernel | The GPU program to execute                       |
| Thread | One logical execution of that program            |
| Block  | A group of threads that can cooperate            |
| Grid   | All blocks submitted for this kernel launch      |
| Warp   | A 32-lane execution group within one block       |
| SM     | The hardware engine that executes resident warps |

Grid ≠ GPU.    Block ≠ SM.    Thread ≠ a dedicated core.

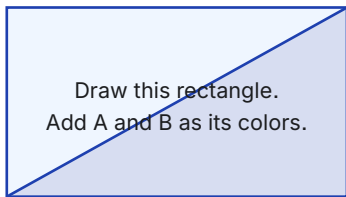
# Keep the work and the machine separate

용어를 마지막으로 정리하겠습니다. 커널은 실행할 프로그램이고, thread는 그 프로그램의 논리적인 실행입니다. block은 서로 협력할 수 있는 thread의 묶음이고, grid는 이번 launch에서 제출한 block 전체입니다. warp는 같은 block의 thread를 32개씩 묶어 명령 실행을 관리하는 단위입니다. 마지막 warp가 꽉 차지 않으면 실제 참여하는 thread는 32개보다 적을 수도 있습니다.

SM은 그 일을 실제로 수행하는 하드웨어입니다. 따라서 grid가 GPU라는 물리 부품에 일대일로 대응하거나, block 하나가 SM 하나를 항상 독점하거나, thread 하나가 특정 코어를 영구적으로 소유하는 것은 아닙니다. 일의 구조와 기계의 구조를 구분하면 처음에 보았던 GPU 메모리와 실행 자원, 그리고 프레임워크 아래에서 벌어지는 일을 훨씬 일관되게 설명할 수 있습니다.

## From two triangles to one line

THEN



NOW

$$C = A + B$$

**The constant idea: apply the same computation to many inputs.**

## From two triangles to one line

처음의 삼각형 두 개로 돌아왔습니다. 예전에는 이 사각형을 그려달라고 말하면서, 각 위치의 색은 A와 B를 더해서 정하라고 해야 했습니다. 지금은 프레임워크에서 C는 A 더하기 B라고 요청하면 됩니다.

그 사이에 GPU가 활용하던 성질 자체가 사라진 것은 아닙니다. 많은 데이터에 같은 계산을 적용한다는 일은 계속 같았습니다. 달라진 것은 계산을 시키기 위해 그림을 그리는 척할 필요가 없어졌고, 그 일을 표현하고 실행하는 체계가 갖춰졌다는 점입니다. 오늘 본 thread와 block, warp와 SM은 그 한 줄과 실제 계산기를 연결하는 언어입니다. GPU를 다음에 사용하실 때 이 구조를 떠올릴 수 있으면 좋겠습니다. 감사합니다.

마지막으로 이 자료의 설명 범위를 말씀드리겠습니다. 오늘 본 커널과 다이어그램은 기본 개념을 이해하기 위한 교육용 예시입니다. 실제 하드웨어의 세부 구조와 메모리 동작, 실행 설정은 GPU 아키텍처와 소프트웨어 버전에 따라 달라질 수 있습니다. 특히 예제의 block 수와 thread 수를 실제 PyTorch 내부의 고정된 구현으로 받아들이지는 않으시면 좋겠습니다. NVIDIA와 Stanford의 자료는 출처를 표시하고 참고한 것이며, 이 발표에 대한 공식적인 보증이나 지지를 뜻하지 않습니다. 설명에 들어간 해석과 오류의 책임은 발표자에게 있습니다. 질문이나 수정 의견은 아래 연락처로 보내주시면 감사하겠습니다.

**Educational scope** This introduction uses simplified kernels and conceptual diagrams. Hardware details, memory behavior, and launch configurations vary by architecture and software version; the examples do not specify PyTorch's actual implementation. NVIDIA and Stanford materials are credited for reference and do not imply endorsement. Interpretations and any errors are the presenter's responsibility.